ASL Recognition Using YOLO and CNN Image Classification

Kevin Oh *
Department of Computer Science
Mount Royal University
koh705@mtroyal.ca

Vincent Ha
Department of Computer Science
Mount Royal University
vha117@mtroyal.ca

Abstract

We developed a machine learning pipeline for real-time pose recognition that maps visual inputs to device controls. Our approach combines hand detection using a fine-tuned object detection model with a custom convolutional neural network for American Sign Language (ASL) classification. The pipeline processes live camera input, isolates hand regions, and maps predicted gestures to keyboard controls. We evaluate this system in the context of video games because, when playing games, input precision and responsiveness provide a revealing test of consistency and accuracy. Beyond gaming, this approach could be helpful in the field of accessibility, enabling custom input mappings for users with disabilities.

1 Introduction

In a growing age of technological dependence, the importance of being able to easily access and use our devices for work, education, or entertainment has become more and more pronounced. Individual needs in terms of what type of input devices they can physically use vary greatly, especially for those suffering from disabilities. We attempt to address this growing concern using a more flexible form of device input, visual data from a camera drawing on inspiration from the pipelines of previous works. Bantupalli and Xie followed a similar multi-model pipeline which inspired us to use multiple types of model in our pipeline (Bantupalli and Xie, 2018). In addition we saw promising results in the YOLO model which encouraged us to use that for object detection in our final pipeline (Li et al., 2022).

While our initial goal was generalized pose/gesture recognition, we refined our scope to prioritize accurate classification of ASL signs. Although the results of our models would change for anything outside of these classes, it still proves as evidence of success for the general pipeline. To this end we used ASL datasets, computer vision library OpenCV, a pre-trained YOLO26n model from Ultralytics finetuned on a hand-keypoint dataset, and a custom CNN Image classification model. Our model pipeline: takes in live camera feed as input, utilizes the YOLO model for creating the boundary box around the hand, the cropped image is then fed into our classification model which is trained to recognize ASL poses. We then detect when certain labels are predicted and map these detections to keyboard inputs to allow the user to give the mapped ASL poses to in-game inputs. This pipeline took a few iterations to finalize in response to the various problems we encountered as we progressed through this experiment.

2 Dataset and Preprocessing

Our dataset Sign Language MNIST (Sign Language MNIST, n.d.) featured clear depictions of each letter with a white background behind it. We attempted to move away from this dataset due to the

*Code available at https://github.com/Kevinmru737/COMP-4630---Final-Project

models created with this dataset being incapable of adapting to more complex backgrounds and angles. This dataset also omits ”J” and ”Z” because those signs require movement and is difficult to capture in a single image. This is not a issue with this project because we are not trying to recognize all ASL sign language, we are instead trying to use the signs as a means for input. So removing 2 out of 26 total signs is not a problem until we require more ways of input.

Figure 1: Example of ”Sign Language MNIST” dataset (source: `https://www.kaggle.com/datasets/datamunge/sign-language-mnist?select=amer_sign2.png`).

Our attempts at getting a better model led us to utilize the ”ASL Alphabet” (Akash Nagaraj, 2018) dataset which features more complex backgrounds and wider variety of positions and angles the hand can be in. This dataset comes with the full alphabet including ”J” and ”Z” along with a sign for ”Space”, ”Del” and ”Nothing”. We have chosen to once again remove ”J” and ”Z” because we do not need that many inputs and to avoid adding more signs that may confuse the model. ”Space” and ”Del” have also been removed for similar reasons although a argument can be made to add them back in due to their distinctness from other hand signs. ”Nothing” has been omitted because YOLO already isolates the image only when a hand appears and will not call on the model if no hand is detected. Therefore there is no need to detect ”Nothing”.

This model unfortunately has worse results compared to our original model. We believe this is due to the complexity of the dataset causing the model to be confused and potentially focus on the wrong aspects of a image to determine a letter. Due to time constraints we have settled with our original model.

In terms of preprocessing, we have focused on resizing the images to a smaller size for easier training. We have also chosen to grayscale the images as we believe that the color of the image should not have that much impact on detection. We have also made sure to grayscale our images when we isolate them before sending to the model to identify. One problem we had with earlier versions of the model is the inability to detect left handed hand signs. Our second model attempted to remedy this by adding a random horizontal flip so that some images can be reminiscent of left handed hand signs. Unfortunately, we have chosen the original model and will be not be applying this transformation.

3 Model Architecture and Training

3.1 Model Selection

We chose a basic CNN architecture as our model for image classification and weren’t overly concerned with the parameters as we were focused on ensuring the overall pipeline is effective. The

CNN had an image converted to single-channel input using 3x3 kernels. Our first convolutional block expanded to a feature map size of 32 and our second convolutional block expanded to 64. The final output layer has 24 classes, which is the English alphabet minus 'j' and 'z'.

Our initial attempts to simply use the CNN on camera-feed input led to the discovery of an extreme dependence on white background colors and having the hand sign take up the entire camera frame. This makes sense given the dataset we were using (see Figure 1). For a lightweight and easy to implement solution, we looked into using a Haar Cascade (fast object-detection algorithm) found on GitHub for detecting hands (Haar cascade for hand detection 2013) . However we found that it was not consistent enough, and upon reading others experiences with YOLO we decided to head in that direction for object detection instead (Li et al., 2022).

3.2 Final Models

YOLO26 is Ultralytics latest in their YOLO series of object detection models. While they have many different types of models, we used their pose models as it seemed the most appropriate for our task of specifically detecting hands. The base models we tried were great at recognizing common objects such as people but did not have a classification for hands. The base pose model detects 17 keypoints (see Figure 2) that are parts of the human body (nose, left eye, etc..), but was indicated as being suitable for a variety of pose estimation tasks (Ultralytics n.d.).

Id	Part
0	nose
1	leftEye
2	rightEye
3	leftEar
4	rightEar
5	leftShoulder
6	rightShoulder
7	leftElbow
8	rightElbow
9	leftWrist
10	rightWrist
11	leftHip
12	rightHip
13	leftKnee
14	rightKnee
15	leftAnkle
16	rightAnkle

Figure 2: Example of YOLO-pose model recognition (modified from: `https://so-development.org/how-to-use-yolo11-for-pose-estimation/`).

We then took this pre-trained model and fine-tuned it with the hand-keypoints dataset from Ultralytics, which gave us bounding boxes on detected hands. This allowed us to essentially use a cropped portion of the image that is centered on detected hands and then feed this cropped image into the classification model. You can see in Figure 3, the pictures have the same ASL sign for the letter F but the YOLO model adds a bounding box, which helps make the correct prediction.

Our final classification model used was our original CNN trained on the Sign Language MNIST dataset (Sign Language MNIST, n.d.). This means that we accepted the limitations of this classification model (requiring certain background colors and nothing but the hand in the camera shot). This decision was made because this model was the most consistent at accurate predictions, even if it required a certain camera set-up. We prioritized the validation of the overall pipeline rather than trying to fix the edge cases which would require extensive classification training.

Another feature of this dataset training was the keypoints you can see that are drawn on the detected hand in Figure 3, however this is unused due to a lack of time.

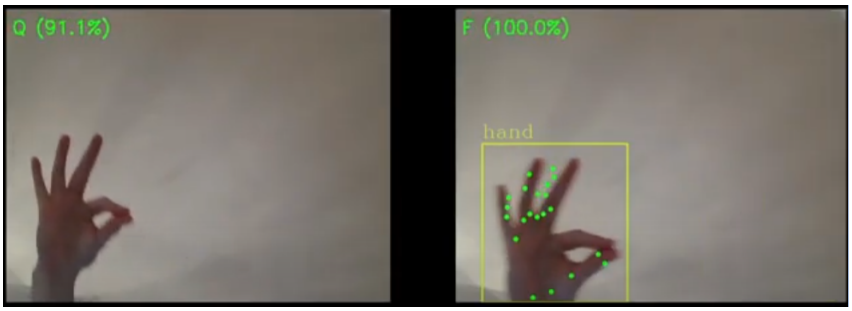

Figure 3: Comparison between with and without YOLO model detection

4 Discussion

4.1 Results

Our final pipeline demonstrates a few key points which we were happy to validate. Firstly that the pipeline of object detection and image classification does work, and secondly that game-inputs are a useful validation metric for tasks that are especially concerned with input precision and consistency. It was immediately apparent when intended inputs were not being recognized, and the speed of recognition was crucial for the game we decided to test it on (a platformer game).

4.2 Lessons

This project showed us how added complexities to dataset can harm image detection and confuse the model. On the other side having a dataset that is hyper-focused will allow for better recognition but poorer adaptability.

4.3 Project Limitations

As is seemingly true when experimenting machine learning models, many limitations of our project come down to time constraints. This does not just cover the training of the models themselves but even regards to what approach is worth pursuing. Although we don't have a polished classification model, we are happy that the overall pipeline seems to be effective, which is what the lion's share of our time was spent validating.

4.4 Future Work

Future work that we would have liked to pursue with this concept would be to generalize the model more. Hands of different skin tones, sizes, and alterations should ideally be recognizable by the model. We would also want to allow for a more lenient recognition of different angles, backgrounds, and potential interferences present in the image. Exploration of the keypoints returned from the YOLO detection model would be an obvious step towards improving generalizability for hand signs and gestures. Along with this we would like to be able to recognize more hand signs along with custom ones. The goal being to eventually allow for a user to give the model a custom hand sign, and it would be able to recognize and tie it to an input.

5 Conclusion

Overall, we believe that using ASL as a base for hand signals as input is a great starting point for generalized hand signs and gestures. If more training is done it should be fairly reasonable to identify each letter and hopefully this can be extrapolated to more hand signs such as digits, other sign languages, or custom signs and gestures.

6 References

Bantupalli, K., and Xie, Y. (2018). American Sign Language recognition using deep learning and computer vision. In Proceedings of the IEEE International Conference on Big Data (BigData), 4896–4899.

Li, T., Yan, Y., and Du, W. (2022). Sign language recognition based on computer vision. In Proceedings of the IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA).

OpenCV (n.d.). Haar cascade for hand detection (2013). https://github.com/Balaje/OpenCV/blob/master/haarcascades/hand.xml

Sign Language MNIST Dataset. (n.d.). Kaggle. https://www.kaggle.com/datasets/datamunge/sign-language-mnist

Ultralytics. (n.d.). Hand keypoints dataset documentation. https://docs.ultralytics.com/datasets/pose/hand-keypoints

Ultralytics. (n.d.). YOLO26 documentation. https://docs.ultralytics.com/models/yolo26/

Akash Nagaraj, ASL Alphabet Dataset. (2018). Kaggle. https://www.kaggle.com/datasets/grassknoted/asl-alphabet/data